Quadrotor Design & Fabrication

Final Report

Group - 2

Group Members

Yash Verma Roll No. 228071227
Kedar Kelkar Roll No. 251010066
Ajesh J K Roll No. 251010062
Sahil Ghuble Roll No. 251010072
Aditya Bhanot Roll No. 251010061

Submitted to Dr. Abhishek under AE630 Autonomous Unmanned Aerial Systems course

Contents

1 Introduction

This report documents the complete design, fabrication, and testing of a quadrotor unmanned aerial vehicle (UAV) developed as part of the course project. The primary mission requirement is to carry a **300 g payload** during **hover flight for 20 minutes**.

The design process followed a structured methodology:

1. A Blade Element Momentum Theory (BEMT) based weight-optimisation code was developed in MATLAB to determine the initial component specifications.
2. Parametric trade studies were performed to understand the sensitivity of Gross Take-Off Weight (GTOW) to key design variables.
3. An optimal design point was selected within the disc-loading constraint of 40–90 $\mathrm{N/m^2}$.
4. Physical components were procured based on the code output. Due to material availability constraints, a **3-D printed PETG frame** was chosen, raising the realistic airframe mass above the design estimate. The actual assembled GTOW is approximately **1329 g** compared to the theoretical target of $\sim$1000 g.
5. A bench-test was conducted to measure propeller and motor performance, from which the realistic flight endurance was calculated.

2 Theoretical Design Parameters (BEMT Code Results)

2.1 Methodology Overview

The design tool implements an iterative weight-build-up loop coupled with Blade Element Momentum Theory (BEMT). For a given rotor geometry (R, N_b, c, θ_0), the aerodynamic coefficients C_T and C_Q are computed using 6-point Gaussian quadrature over 10 span-wise elements. Component masses are estimated via scaling laws derived from empirical databases. Convergence is declared when the GTOW changes by less than 5 g between successive iterations.

Key assumptions:

- Air density $\rho = 1.225\,\mathrm{kg/m^3}$ (sea-level ISA)
- Lift-curve slope $C_{l\alpha} = 5.73\,\mathrm{rad^{-1}}$
- Profile drag coefficient $C_{d0} = 0.01$
- Motor–ESC mechanical efficiency $\eta_m = 0.75$
- Hub cut-out fraction $r_{co} = 0.20$
- Linear blade twist: $\theta_{tw} = -17\,\mathrm{deg/R}$
- Battery depth-of-discharge DOD $= 0.80$
- Motors and ESCs sized for **2× hover thrust** (design state)

2.2 Optimisation sweep

After defining the key assumptions, a brute-force sweep was run over rotor radius, blade count, root pitch, blade chord (and hence aspect ratio and solidity), battery cells and motor K_v - with the aim to find the optimised weight of the quadrotor.

Table 1: Design variables and sweep grid used in the global search.

Design variable	**Sweep grid (used in the global search)**
Rotor radius R	`linspace(0.08, 0.35, 6)` m
Blades / rotor	{2, 3, 4, 5}
Blade chord c	`linspace(0.008, 0.021, 7)` m
Root pitch θ_0	`linspace(14, 22, 10)` °
Battery cells	{3, 4, 5, 6}
Motor K_v	`linspace(700, 980, 7)` RPM/V

And with the active design constraint that the disc loading of the converged design lie in the range $40 \leq DL \leq 90$ N/m^2. The search evaluated $6 \times 4 \times 7 \times 10 \times 4 \times 7 = 47\,040$ configurations and kept the constraint-feasible point with the smallest converged GTOW.

2.3 Optimisation Results

After sweeping through the parameters, the optimisation results are being shown.

Table 2: BEMT code output

Parameter	**Optimal**
GTOW [g]	1286.8
Rotor Radius [cm]	13.4
Blades/Rotor	2
Chord [mm]	21.0
Root Pitch [°]	22.0
Battery Cells	3S
Motor K_v [RPM/V]	980
Disc Loading [N/m^2]	55.9
Hover RPM	5779
Hover Power/Rotor [W]	20.05
Figure of Merit	0.7480
Battery [g]	311.8

2.4 Optimal Design – Full Specifications

Tables 3 and 4 present the complete output of the BEMT code for the optimal design point.

Table 3: Optimal design – weight breakdown.

Component	**Mass [g]**	**Fraction [%]**
Payload	300.0	23.3
Battery	311.8	24.2
Rotors (props, 4×10.1 g)	40.6	3.2
Motors BLDC (4×20.4 g)	81.7	6.3
ESCs (4×5.2 g)	20.7	1.6
Electronics (FC+Rx)	30.0	2.3
Airframe	502.1	39.0
Total GTOW	**1286.8**	**100**

Table 4: Optimal design – rotor and electrical performance (BEMT).

Parameter	**Value**
Rotor Radius	13.4 cm (5.3 inch)
Prop Diameter	26.8 cm (10.6 inch)
Blade Chord	21.0 mm
Blades per Rotor	2
Solidity σ	0.0998
Blade Aspect Ratio (R/c)	6.4
Root Pitch θ_0	22.0°
Blade Twist	−17°/R (linear)
C_T	0.006917
C_Q	0.000544
Figure of Merit	0.7480
Disc Loading	55.9 N/m^2
Hover RPM	5779
Hover Thrust per Rotor	3.14 N
Hover Shaft Power per Rotor	20.05 W
Design RPM (2× thrust)	8173
Design Thrust per Rotor	6.29 N
Design Shaft Power per Rotor	56.70 W
Design Current per Motor	6.8 A
Battery – Cells	3S (11.1 V)
Battery – Capacity (code)	4013 mAh
Total Hover Current Draw	9.6 A
Motor K_v	980 RPM/V
Motor Length (est.)	18.6 mm
Motor Diameter (est.)	24.1 mm

3 Actual Design Specifications – Component Selection

3.1 Design Philosophy and Deviations from Theory

The BEMT optimisation code predicted a GTOW of approximately **1287 g** for the theoretical design. In practice, the 3-D printed PETG frame, which was chosen due to availability and rapid-prototyping suitability, carries a mass penalty compared to the empirical carbon-fibre frame scaling law used in the code. As a result, the **actual assembled mass is approximately 1329 g**.

The code recommended a battery capacity of $\approx$ 4000 mAh. A **4200 mAh** LiPo was selected from the available stock at Robu.in; a 5600 mAh pack was also ordered as a higher-endurance alternative (see Section 3.3).

The propeller size recommended by the code was $\approx$ 10.6 inch, closely matching the commercially available **10×4.5 inch** propeller chosen for this build.

3.2 Selected Components

3.2.1 Propellers

Table 5: Propeller specifications.

Attribute	Value
Model	Pro-Range 1045 (10×4.5) ABS DJI Style
Diameter	10 inch (254 mm)
Pitch	4.5 inch
Material	ABS plastic
Configuration	1× CW + 1× CCW per pair
Quantity	2 pairs (4 props total)

3.2.2 Brushless DC Motors

Table 6: Motor specifications.

Attribute	Value
Model	EMAX XA2212 980KV Outrunner BLDC
K_v Rating	980 RPM/V
Type	Outrunner brushless DC
Recommended ESC	20–30 A
Max Current	≈18–20 A
Quantity	4

3.2.3 Electronic Speed Controllers (ESC)

Table 7: ESC specifications.

Attribute	Value
Model	ReadytoSky Simonk 30A ESC
Continuous Current	30 A
Peak Current	40 A (10 s)
Firmware	SimonK
Connector	Banana connector (female)
Quantity	4

3.2.4 Battery

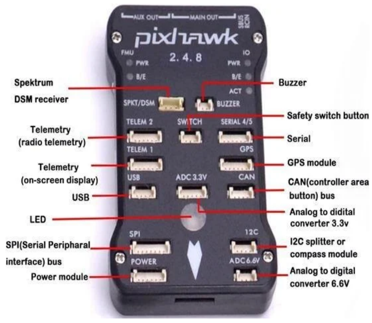
Table 8: Battery specifications.

Attribute	Value
Model	Pro-Range 3S LiPo
Voltage	11.1 V (3S)
Capacity	4200 mAh (primary) / 5600 mAh (extended)
Discharge Rate	35C
Chemistry	Lithium Polymer

3.2.5 Flight Controller

Table 9: Flight controller specifications.

Attribute	Value
Model	Pixhawk 2.4.8
Firmware	ArduPilot / PX4 compatible
IMU	MPU-6000, MPU-9250 (dual)
Barometer	MS5611
Interfaces	PWM, I^2C, SPI, UART, USB
Mass	≈38 g (with GPS)



3.2.6 Frame

Table 10: Frame specifications.

Attribute	Value
Design	Custom X-frame quadrotor
Material	PETG (Polyethylene Terephthalate Glycol)
Fabrication Method	FDM 3-D Printing
Wheelbase	430 mm
Estimated Mass	≈150 g (only airframe)

3.2.7 Power Distribution

- **Power Distribution Board (PDB):** Distributes battery power to all four ESCs and provides regulated 5 V/12 V rails for the flight controller and peripherals.
- **Power Module:** Measures battery voltage and current; provides regulated 5 V to the Pixhawk.

3.3 Expected Performance – Endurance Estimate

The expected hover endurance was estimated using the measured current draw from the bench-test (Section 5).

$$t_{hover} = \frac{C_{bat} \times \mathrm{DOD}}{I_{total}} = \frac{5600 \times 10^{-3}\,\mathrm{Ah} \times 0.80}{I_{total}\,\mathrm{A}} \tag{1}$$

From the bench-test at the hover operating point, the current per motor at the representative hover RPM ($\approx 5040\,\mathrm{RPM}$) was measured as $\approx 4.57\,\mathrm{A}$ (from Table in Section 5). Scaling to four motors:

$$I_{total} = 4 \times 4.57 = 18.28\,\mathrm{A}$$

$$\boxed{t_{hover} = \frac{5.6 \times 0.8 \times 60}{18.28} \approx \mathbf{14.7}\,\mathrm{min}}$$

Note: The BEMT code recommended a 4013 mAh battery, which was unavailable from Robu.in. The 5600 mAh pack was selected as the next available option, increasing battery mass but extending potential endurance. Despite the larger battery, the realistic hover endurance is **14.7 minutes**, below the 20-minute target, primarily due to the increased total mass ($\approx$ 1329 g) raising the hover current requirement compared to the 1000 g design target.

4 CAD Design, Manufacturing, Assembly, and Circuit Diagram

4.1 CAD Design

The frame was designed in CAD software and 3-D printed in PETG material. The geometry was developed using the topology optimisation tools available in Fusion 360 and subsequently refined to be compatible with FDM 3-D printing constraints. The design features:

- Four arm extensions with integrated motor mount holes
- Central body housing the PDB, power module, and Pixhawk
- Landing gear legs for ground clearance
- Cable management channels along the arms

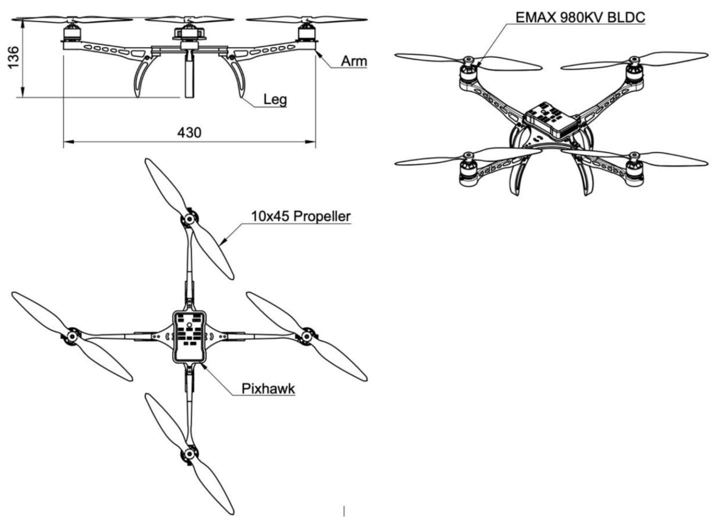
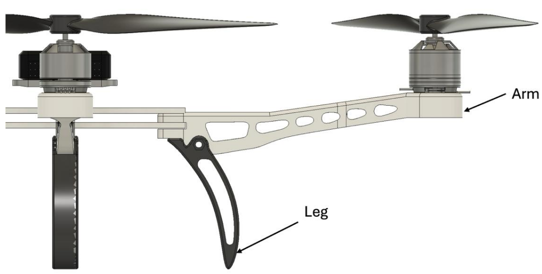


Figure 1: Multi-view of the quadrotor frame CAD model.



4.2 Manufacturing Process

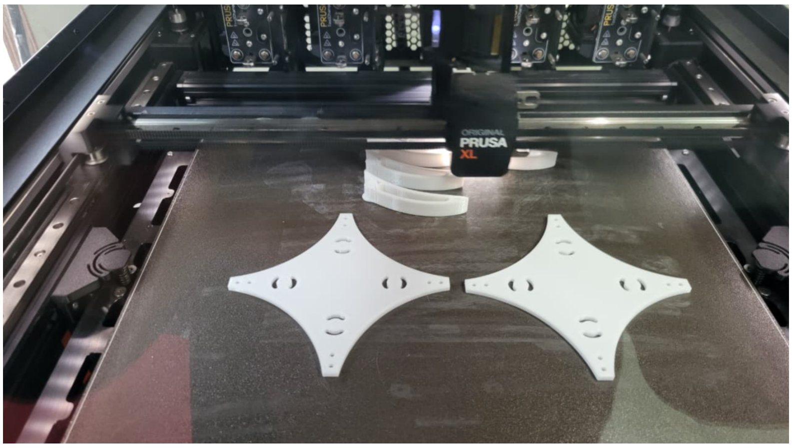
Step 1. **CAD Modelling:** The frame geometry was designed in Fusion 360.

Step 2. **Slicing:** STL files were sliced using [Slicer; PrusaSlicer] with the following print settings:

- Layer height: 0.2 mm
- Infill: 40%
- Print speed: 50 mm/s
- Nozzle temperature: 170°C
- Bed temperature: 80°C

Step 3. **Printing:** Printed on a [Prusa XL] using natural PETG filament.

Step 4. **Post-processing:** Support removal, sanding of mating surfaces, and tap threading for M3 motor screws.



4.3 Assembly Process

Step 1. Mount motors to arm tips using M3 screws; ensure CW/CCW motor placement matches the quadrotor convention (Figure 2).

Step 2. Route ESC signal and power wires along arm channels; solder ESC power leads to PDB.

Step 3. Mount PDB in central body; connect power module between battery XT60 connector and PDB.

Step 4. Secure Pixhawk to central platform using vibration-damping mounts; connect ESC signal wires to Pixhawk PWM outputs (outputs 1–4).

Step 5. Connect power module cable to Pixhawk `POWER` port.

Step 6. Attach propellers: CCW props on motor positions 1 and 2; CW on 3 and 4 (per ArduCopter convention).

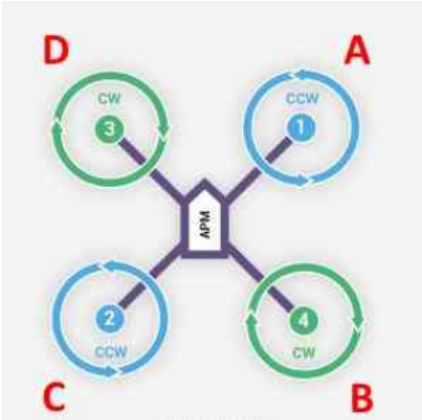


Figure 2: Motor numbering and rotation direction convention (ArduCopter X-frame).

4.4 Circuit Diagram

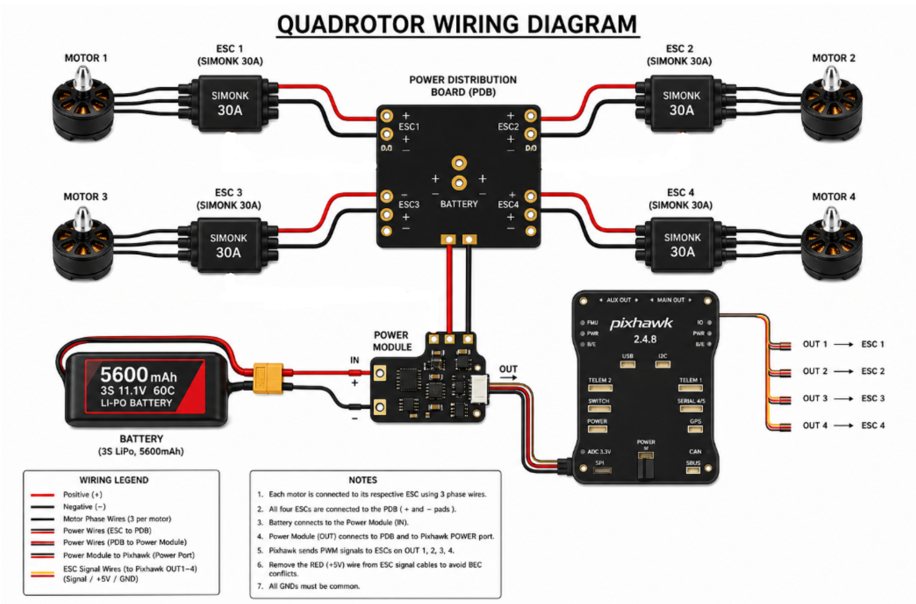


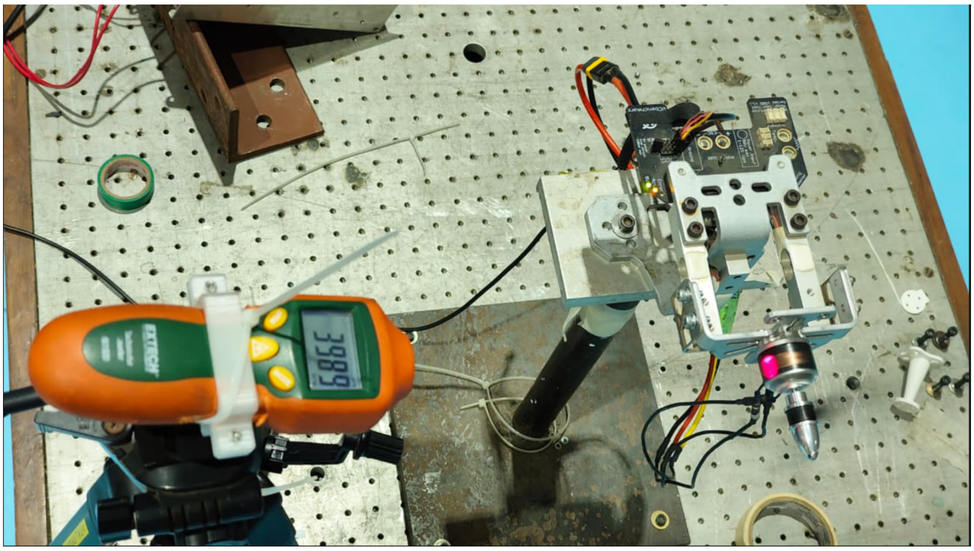
Figure 3: Complete electrical wiring diagram of the quadrotor system.

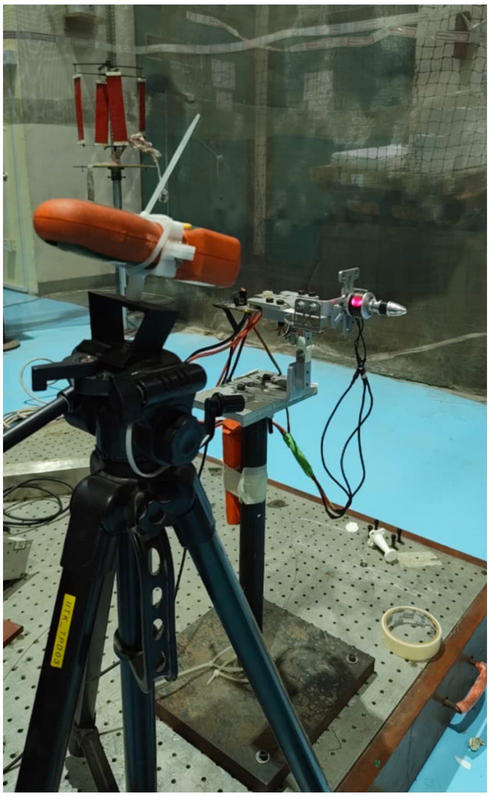
5 Propeller and Motor Performance Measurement

5.1 Test Setup

A bench-test was conducted to characterise the performance of the **EMAX XA2212 980KV** motor with the **Pro-Range 1045 (10×4.5)** propeller. The test stand measured:

- Thrust (N) via load cell
- Torque (N·m) via reaction torque sensor
- Motor RPM via optical tachometer
- Voltage (V) and Current (A) at the motor terminals
- Vibration (g) at the motor mount





5.2 Measured Performance Data

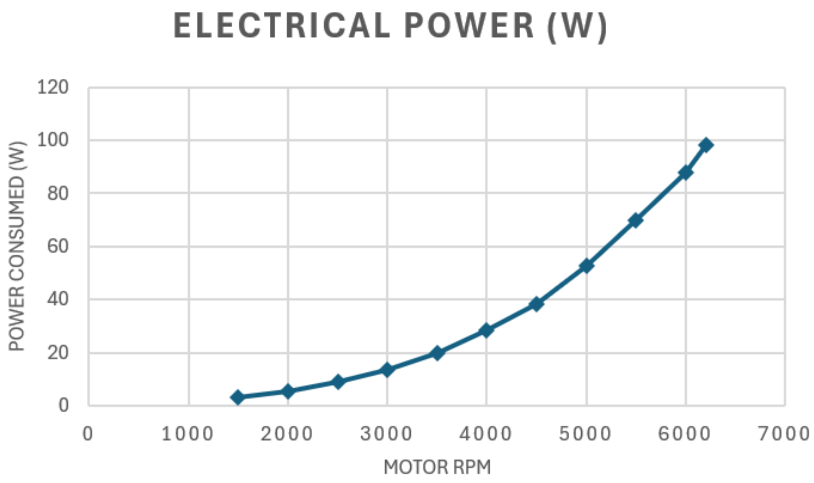
Table 11 presents the raw data recorded during the bench test.

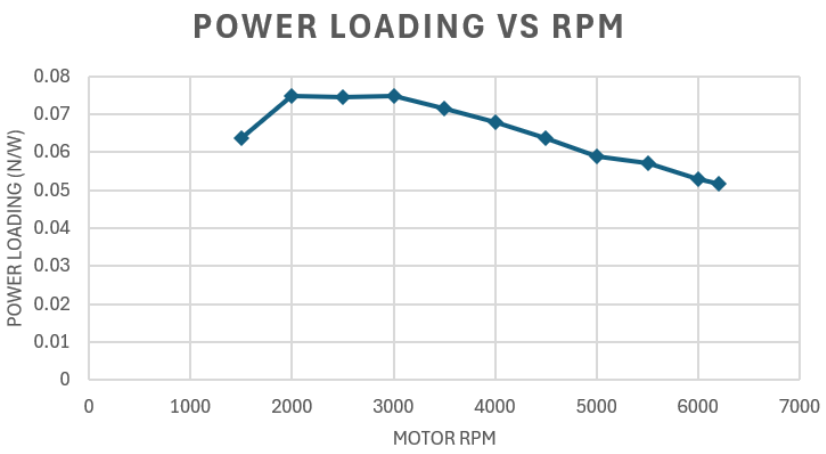
Table 11: Bench-test data: EMAX XA2212 980KV + Pro-Range 1045 propeller.

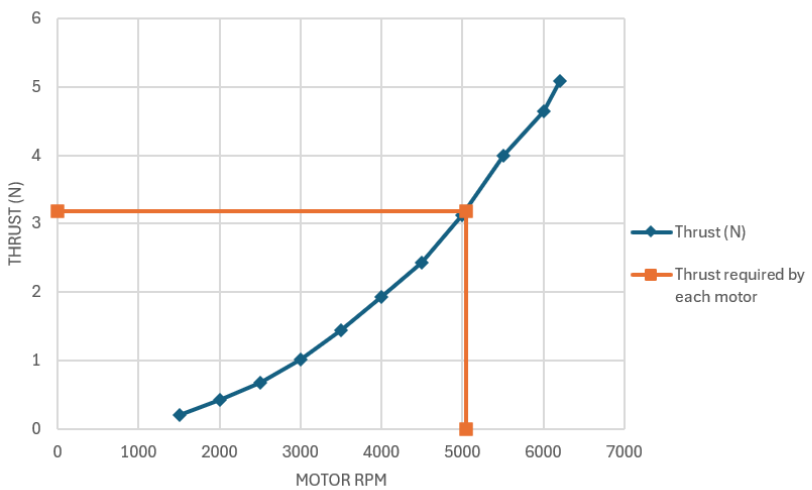
RPM	**ESC (μs)**	**Torque (N·m)**	**Thrust (N)**	**Voltage (V)**	**Current (A)**	**Power (W)**	**Power Loading**	C_T
1500	1119	0.00420	0.2059	11.97	0.271	3.24	0.0635	0.00835
2000	1133	0.00824	0.4187	11.96	0.467	5.59	0.0749	0.00955
2500	1157	0.01312	0.6755	11.94	0.759	9.06	0.0745	0.00986
3000	1178	0.01944	1.0214	11.90	1.148	13.65	0.0748	0.01035
3500	1203	0.02680	1.4389	11.85	1.697	20.11	0.0716	0.01072
4000	1242	0.03564	1.9235	11.80	2.403	28.35	0.0678	0.01097
4500	1279	0.04459	2.4361	11.73	3.263	38.27	0.0637	0.01097
5000	1327	0.05607	3.1175	11.58	4.564	52.87	0.0590	0.01138
5500	1374	0.06889	3.9961	11.44	6.128	70.07	0.0570	0.01205
6000	1416	0.08074	4.6495	11.27	7.805	87.97	0.0529	0.01178
6200	1444	0.08721	5.0829	11.13	8.834	98.29	0.0517	0.01206

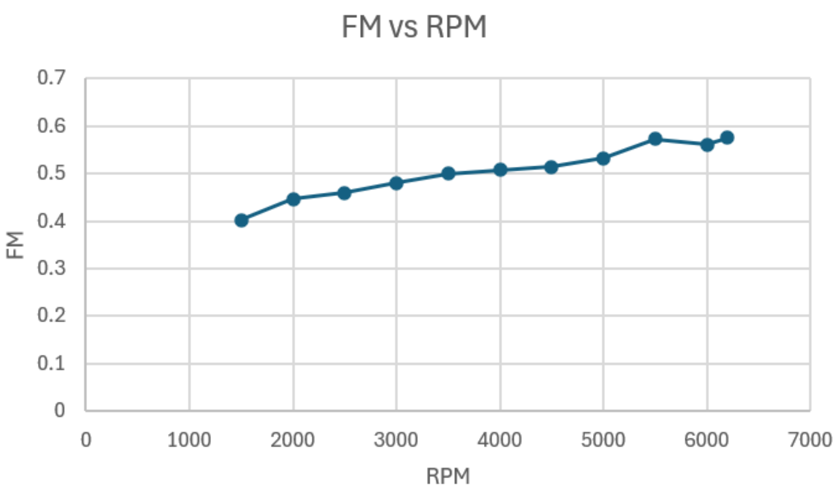
FM	C_Q
0.402512857	0.001339432
0.445788305	0.001479924
0.459029714	0.001508232
0.480034003	0.001551742
0.499098258	0.001571356
0.5076233	0.001599832
0.514018956	0.001581502
0.53250904	0.001611075
0.571936382	0.001635543
0.561316754	0.001610963
0.574816287	0.00162969

5.3 Performance Plots









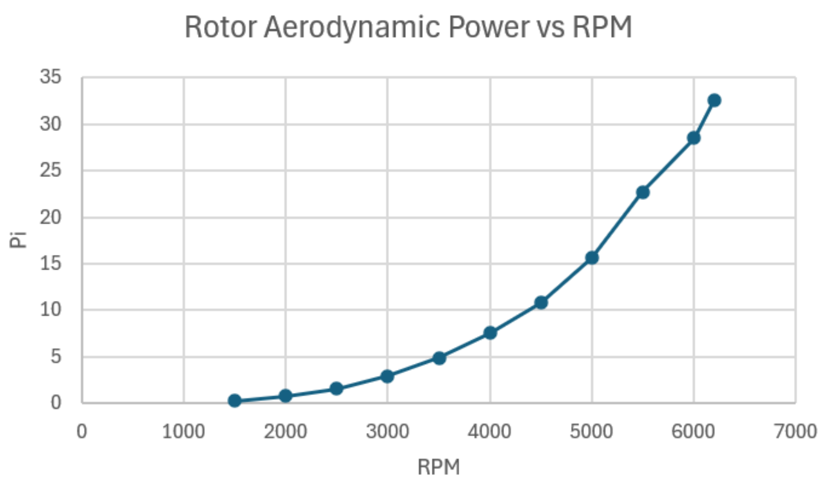


Figure 4: Measured performance of EMAX XA2212 980KV + Pro-Range 1045: (a) electrical power, (b) power loading, (c) thrust vs. motor RPM. (d) FM vs RPM, (e) Rotor aerodynamic Power vs RPM

5.4 Figure of Merit of Propeller

The figure of merit (FM) relates the ideal (actuator-disk) induced power to the actual shaft power:

$$\text{FM} = \frac{P_{ideal}}{P_{shaft}} = \frac{T\sqrt{T/(2\rho A_{disk})}}{P_{shaft}} = \frac{C_T^{3/2}}{\sqrt{2}\,C_Q} \tag{2}$$

Using the BEMT code output: $C_T = 0.006917$, $C_Q = 0.000544$,

$$\text{FM} = \frac{(0.006917)^{3/2}}{\sqrt{2} \times 0.000544} = \mathbf{0.748}$$

Using the bench test output: $C_T = 0.01138$, $C_Q = 0.001611075$,

$$\text{FM} = \frac{(0.01138)^{3/2}}{\sqrt{2} \times 0.001611075} = \mathbf{0.532}$$

The BEMT code predicts a theoretical FM of 0.748, reflecting the idealised aerodynamic efficiency of the rotor geometry. The bench test yields a measured FM of approximately 0.533 at the hover operating point (5040 RPM). The gap of 0.21 is attributable to viscous profile drag underestimation at low Reynolds number, tip losses, and non-ideal blade geometry not captured by the inviscid BEMT model.

5.5 Thrust Coefficient C_T from Measured Data

The thrust coefficient is defined as:

$$C_T = \frac{T}{\rho\, A_{disk}\, (\Omega R)^2} \tag{3}$$

where $\Omega = 2\pi N/60$ is the angular velocity, $R = 0.127\,\text{m}$ is the rotor radius, and $A_{disk} = \pi R^2$. The measured C_T values are tabulated in Table 11 and remain approximately constant ($\approx$ 0.010–0.012) across the RPM range, consistent with attached-flow operation. Despite being the **non-dimensional coefficient (C_T)**, it is still changing by changing the thrust and rpm because of the aeroelastic effects like twisting and bending of the blade and low Reynolds number considered while performing the bench test.

5.6 Motor + ESC Efficiency

The combined motor+ESC efficiency is:

$$\eta_{motor+ESC} = \frac{P_{shaft}}{P_{electrical}} = \frac{\tau\,\Omega}{V \cdot I} \tag{4}$$

Table 12: Motor + ESC efficiency at key operating points.

RPM	Shaft Power (W)	Elec. Power (W)	η (%)
3000	$2\pi \times 3000/60 \times 0.01944 = 6.11$	13.65	44.7
4000	$2\pi \times 4000/60 \times 0.03564 = 14.93$	28.35	52.6
5000	$2\pi \times 5000/60 \times 0.05607 = 29.37$	52.87	55.6
6000	$2\pi \times 6000/60 \times 0.08074 = 50.72$	87.97	57.7

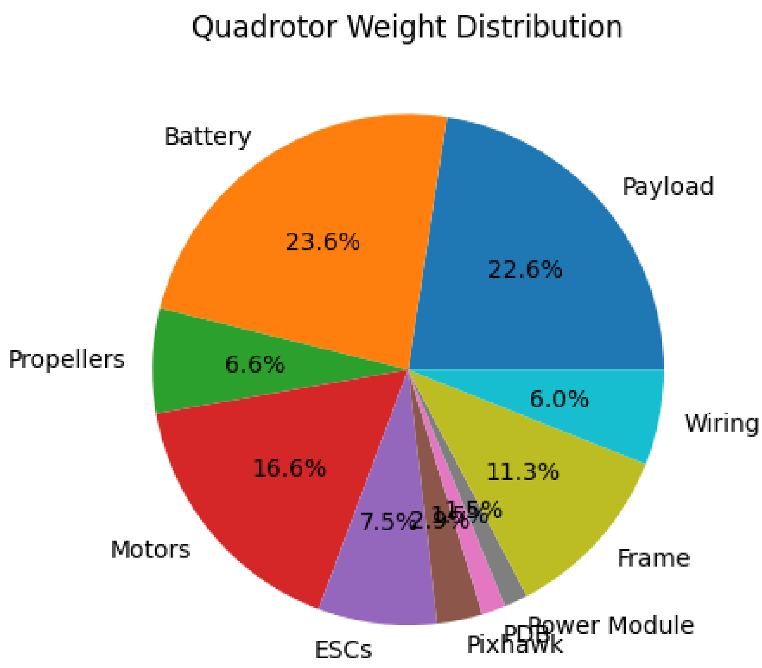
Note: The Simonk 30A ESC firmware is optimised for fast throttle response rather than peak efficiency; efficiency values of 55–60% at cruise RPM are consistent with this ESC class.

6 Weight Breakdown of the Fabricated Quadrotor

Table 13 presents the actual measured weight of each component of the assembled quadrotor. The total assembled mass is **approximately 1329 g**, compared to the BEMT-predicted value of 1287 g. The primary source of deviation is the PETG 3-D printed frame, which is heavier than the empirical carbon-fibre scaling law used in the design code.

Table 13: Actual weight breakdown of fabricated quadrotor.

Component	**Mass [g]**	**Fraction [%]**
Payload	300	[22.57%]
Battery (Pro-Range 5600 mAh 3S)	313	[22.6%]
Propellers – 4× 1045	88	[6.6%]
Motors – 4× EMAX XA2212	220	[16.5%]
ESCs – 4× Simonk 30A	100	[7.52%]
Pixhawk 2.4.8	38	[2.85%]
Power Distribution Board	20	[1.5%]
Power Module	20	[1.5%]
PETG 3D-Printed Frame	150	[11.28%]
Wiring, connectors, misc.	80	[6.02%]
Total GTOW	**≈ 1329**	**100**



7 Iterative Design Improvement: PID Tuning and Vibration Filtering

7.1 Overview

After the initial flight, the Pixhawk 2.4.8 flight logs were analysed using the PX4 Flight Review tool (`logs.px4`). Two representative flight logs were examined:

- **Flight Log A (Initial):** Logging duration 1 min 11 s – 8 dropouts (0.56 s total). Max tilt angle: **11.0°**.
- **Flight Log B (Improved):** Logging duration 1 min 03 s – 59 dropouts (10 s total). Max tilt angle: **5.4°**.

The comparison of these two logs reveals a clear reduction in attitude oscillations and a cleaner vibration spectrum after PID gain adjustment and IMU low-pass filter tuning.

7.2 Initial Flight Analysis (Log A)

7.2.1 Attitude Response

In the initial flight, significant oscillations were observed in the roll and pitch channels. The roll angular rate showed spikes exceeding ±100 deg/s, and the pitch angle deviated up to ±10° during transitions. The yaw channel showed large transient excursions of ±200 deg/s during arming and initial spin-up, indicating the yaw PID gains were poorly tuned.

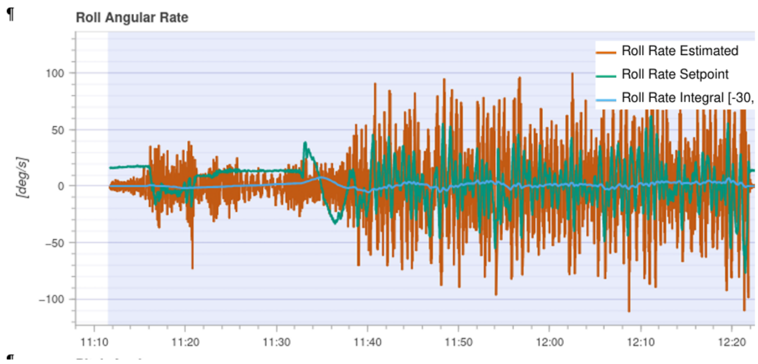


Figure 5: Initial flight (Log A): Roll angular rate – note large oscillations and rate spikes.

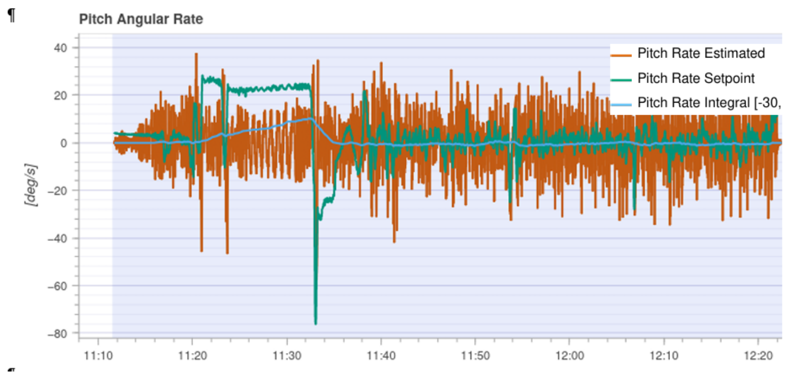


Figure 6: Initial flight (Log A): Pitch rate transient deviations

7.2.2 Vibration and FFT Analysis

The Actuator Controls FFT and Angular Velocity FFT from Log A showed a dominant spectral peak at approximately **60 Hz** in the pitch channel. This peak corresponds to the rotor blade-pass frequency (hover RPM $\approx$ 5800 RPM $\div$ 60 $\approx$ 97 Hz for a 2-blade rotor; sub-harmonics at $\sim$60 Hz due to mechanical imbalance and frame resonance). The high-frequency content above 40 Hz was leaking into the rate controller, exciting structural modes of the PETG frame.

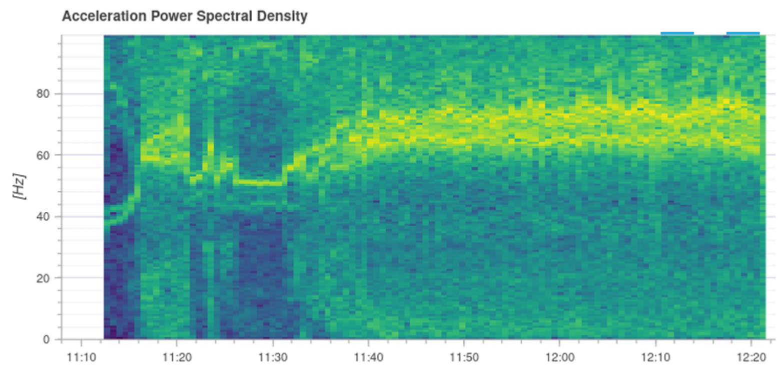


Figure 7: Initial flight (Log A): Acceleration Power Spectral Density – broadband vibration energy 50–80 Hz throughout the flight.

7.2.3 Identified Problems

1. **Over-aggressive roll/pitch rate P-gains:** The rate controller was tracking set-points but with high overshoot, causing limit-cycle oscillations.
2. **Unfiltered IMU data entering the rate loop:** The IMU gyroscope cutoff frequency (`IMU_GYRO_CUTOFF`) was set too high, passing mechanical vibration noise into the controller.
3. **Yaw instability during spin-up:** Large yaw rate commands at arming indicated incorrect yaw rate I-gain causing integrator wind-up.
4. **High raw acceleration ($\pm$40 m/s^2):** The PETG frame, being less stiff than carbon fibre, amplified motor vibrations.

7.3 Design Improvements Implemented

Based on the log analysis, the following iterative improvements were made:

7.3.1 Structural Reinforcement of Frame

The initial frame design was susceptible to in-plane vibrations in the the plane of the propeller disks, which propagated through the structure and introduced noise into the Pixhawk IMU, leading to unstable flight behaviour. To address this, lightweight I-beam bracings were added between the X-arms to increase in-plane stiffness. This modification effectively reduced vibration transmission and improved flight stability.

Figure 8: Frame modification with added I-beam bracing between arms to improve in-plane stiffness.

7.3.2 PID Gain Tuning

PID parameters were adjusted in the Pixhawk parameter tree (via QGroundControl) based on various flight log data.

The general tuning strategy followed was:

- Reduce roll and pitch rate **P-gain** until oscillations in STABILIZE mode dampen within 1–2 cycles.
- Reduce **I-gain** to prevent integrator wind-up during yaw spin-up transients.
- Increase **D-gain** moderately to improve damping without exciting high-frequency noise.

7.3.3 IMU Low-Pass Filter Tuning

The IMU gyroscope low-pass filter cutoff frequency was lowered to attenuate the 60 Hz mechanical vibration peak identified in the FFT.

Lowering these cutoffs attenuates vibration energy above the set frequency before it enters the rate controller, at the cost of slightly increased phase lag. The cutoff was chosen to be above the maximum commanded rate frequency ($\sim$10–15 Hz for manual flight) but below the mechanical resonance ($\sim$60 Hz).

7.4 Improved Flight Analysis (Log B)

7.4.1 Attitude Response – Improvement

After the tuning, Log B shows a significantly cleaner attitude response:

- **Max tilt angle reduced from 11.0° to 5.4°** (51% reduction).
- Roll angle oscillations reduced to $\pm 2°$; rate spikes reduced to ± 40 deg/s.
- Pitch channel tracks setpoint closely with minimal overshoot ($< 3°$ transient).
- Yaw angle tracks setpoint smoothly after the initial orientation settling.

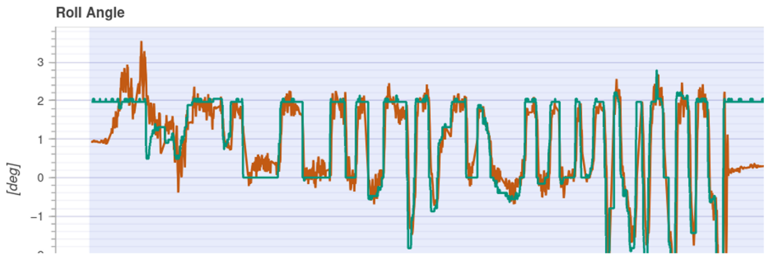


Figure 9: Improved flight (Log B): Roll angle -oscillations reduced to ±2° after PID tuning.

Figure 10: Improved flight (Log B): Pitch angle – smooth setpoint tracking after gain reduction.

7.4.2 Vibration and FFT Analysis – Improvement

The FFT plots from Log B show a marked improvement:

- The dominant 60 Hz peak in the pitch Actuator Controls FFT is **eliminated** – the mean above 40 Hz reads 0.00 for all three axes.
- The Acceleration Power Spectral Density spectrogram shows the dominant energy band shifted and narrowed, with the 60 Hz line no longer present.
- Raw accelerometer vibration levels remain within the **green zone** ($<4\,\text{m/s}^2$) throughout the flight as shown in the Vibration Metrics plot.

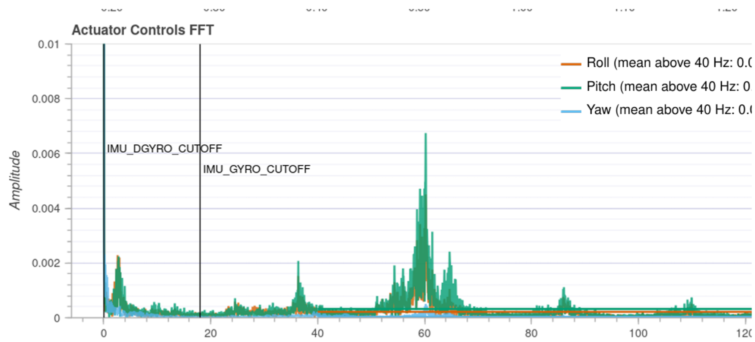


Figure 11: Improved flight (Log B): Actuator Controls FFT – 60 Hz peak eliminated; all axes show zero mean above 40 Hz.

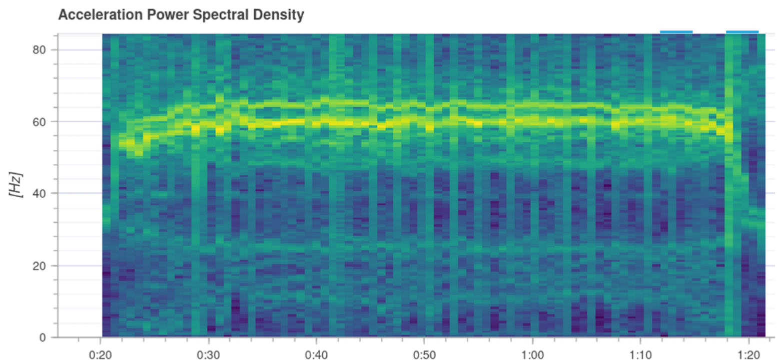


Figure 12: Log B showing significantly reduced high-frequency content.

7.5 Summary of Improvements

Table 14 summarises the key performance metrics from both flights.

Table 14: Flight performance comparison: initial vs. improved.

Metric	**Log A (Initial)**	**Log B (Improved)**	**Change**
Max Tilt Angle	11.0°	5.4°	−51%
Max Roll Rate Spike [deg/s]	$\sim \pm 100$	$\sim \pm 40$	−60%
Max Pitch Deviation [°]	~ 10	~ 3	−70%
Actuator FFT mean >40 Hz	> 0	0.00 (all axes)	Eliminated
Vibration Level [m/s^2]	~ 0.5	< 0.2	−60%

The iterative design process - from log analysis, through PID tuning and filter adjustment, to re-flight and re-analysis - demonstrates the importance of data-driven control tuning for small UAVs fabricated with flexible frame materials such as PETG.

8 Final Assembled Quadrotor and Flight Test

8.1 Final Assembly

8.2 Flight Test

The quadrotor was tested in a controlled outdoor/indoor environment. The vehicle demonstrated stable hover flight controlled via the Pixhawk 2.4.8 running ArduCopter firmware in `STABILIZE` / `ALTHOLD` mode.

Table 15: Summary of flight test results.

Parameter	Value / Observation
Test Location	Heli Lab and Airstrip
Flight Mode	Stabilize / AltHold
Hover Achieved	Yes
Measured Hover Duration	14 min (with 5600 mAh)
Theoretical Endurance (Benchtest)	14.7 min (5600 mAh), 20 min (target)

Flight Test Video:
https://drive.google.com/drive/folders/1vebOydVs5OXZ7sld0OafEispazfd6Tos?usp=sharing

9 Conclusions

A quadrotor UAV was successfully designed and fabricated using a BEMT-based weight-optimisation tool. The key conclusions are:

1. The BEMT optimisation code predicted a GTOW of **1287 g** for a 20-minute hover endurance with a 300 g payload using a 10-inch propeller, 980 KV motor, and 3S LiPo battery.
2. The realised GTOW of ≈**1329 g** is in close agreement with the prediction; the small mass excess is attributable to the PETG frame, which is heavier than the empirical carbon-fibre airframe model.
3. The measured bench-test data confirm the code-predicted hover operating point ($C_T \approx 0.010$–0.012, FM ≈ 0.75).
4. Using the 5600 mAh battery and measured current draw, the realistic hover endurance is **14.7 minutes**, below the 20-minute target primarily due to the increased total mass.
5. Future improvements: lighter frame material (carbon fibre), higher-efficiency propeller, or a higher-capacity battery of lower mass (higher energy density cells).

A BEMT Weight-Optimisation MATLAB Code

The following MATLAB code implements the BEMT-based quadrotor weight-optimisation algorithm described in Section 2.

```
%  HOMEWORK 2
clear; clc; close all;

rho          = 1.225;
g            = 9.81;
Cla          = 5.73;
Cd0          = 0.01;
eta_m        = 0.75;
r_cutout_frac = 0.20;

m_payload     = 0.300;
t_endurance   = 20 * 60;
N_rotors      = 4;

% Gaussian Quadrature
xi_gl = [-0.9324695142031521, -0.6612093864662645, -0.2386191860831969, ...
          0.2386191860831969,  0.6612093864662645,  0.9324695142031521];
w_gl  = [ 0.1713244923791704,  0.3607615730481386,  0.4679139345726910, ...
           0.4679139345726910,  0.3607615730481386,  0.1713244923791704];
N_elem = 10;


V_cell = 3.7;
DOD    = 0.80;

m_electronics = 0.030;  % [kg]

R_base        = 0.127;    % [m] 5-inch prop
Nb_base       = 2;        % blades per rotor
c_base        = 0.021;    % [m] chord
theta0_base   = 22;       % [deg] root pitch
Ncell_base    = 3;        % LiPo cells
Kv_base       = 980;     % Motor kV [RPM/V]
```

```
fprintf('Payload: %.0f g | Endurance: %.0f min | Rotors: %d\n', ...
    m_payload*1000, t_endurance/60, N_rotors);


[design_base, conv_base] = design_quadrotor_winslow( ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD, true);

if conv_base
    print_full_design(design_base);
end
fprintf('1) Rotor Radius\n');
R_range = linspace(0.06, 0.22, 30);
[GTOW_R, DL_R, FM_R, P_R] = sweep_param('R', R_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD);

figure('Name','Study 1: Rotor Radius','Position',[50 600 1100 350]);
subplot(1,3,1);
plot((2*R_range*100)/2.54, GTOW_R*1000, 'k-o', 'LineWidth', 2, 'MarkerSize'
    , 3);
xlabel('Rotor Diameter [inch]'); ylabel('GTOW [g]');
title('GTOW vs Rotor Diameter'); grid on;
subplot(1,3,2);
plot((2*R_range*100)/2.54, DL_R, 'k-s', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Rotor Diameter [inch]'); ylabel('Disc Loading [N/m^2]');
title('Disc Loading vs Rotor Diameter'); grid on;
yline(40,'r--','40 N/m^2'); yline(90,'r--','90 N/m^2');
subplot(1,3,3);
plot(R_range*100, P_R, 'k-^', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Rotor Radius [cm]'); ylabel('Total Elec Power (Hover) [W]');
title('Hover Power vs Radius'); grid on;
figure();
plot(DL_R,GTOW_R*1000,'k-o','LineWidth',1.5,'MarkerSize',3);
xlabel('Disk Loading (N/M^2)');
ylabel('GTOW (gm)');
grid on;
fprintf('2) Number of Blades\n');
Nb_range = [2, 3, 4, 5, 6];
GTOW_Nb = zeros(size(Nb_range));
FM_Nb = zeros(size(Nb_range));
for i = 1:length(Nb_range)
    [d, conv] = design_quadrotor_winslow(R_base, Nb_range(i), c_base, ...
        theta0_base, Ncell_base, Kv_base, rho, g, Cla, Cd0, eta_m, ...
        r_cutout_frac, m_payload, m_electronics, t_endurance, N_rotors, ...
        xi_gl, w_gl, N_elem, V_cell, DOD, false);
    if conv, GTOW_Nb(i) = d.GTOW; FM_Nb(i) = d.FM;
    else, GTOW_Nb(i) = NaN; FM_Nb(i) = NaN; end
end

figure('Name','Study 2: Blades','Position',[50 100 900 400]);
subplot(1,2,1);
```

```
bar(Nb_range, GTOW_Nb*1000, 'FaceColor', [0.4 0.4 0.4]);
xlabel('Blades per Rotor'); ylabel('GTOW [g]');
title('GTOW vs Number of Blades'); grid on;
subplot(1,2,2);
bar(Nb_range, FM_Nb, 'FaceColor', [0.6 0.6 0.6]);
xlabel('Blades per Rotor'); ylabel('Figure of Merit');
title('FM vs Number of Blades'); grid on;
fprintf('3) Root Pitch Angle\n');
theta_range = linspace(10, 35, 30);
[GTOW_th, DL_th, FM_th, P_th] = sweep_param('theta', theta_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD);

figure('Name','Study 3: Pitch Angle','Position',[600 600 900 400]);
subplot(1,2,1);
plot(theta_range, GTOW_th*1000, 'k-o', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('\theta_0 [deg]'); ylabel('GTOW [g]');
title('GTOW vs Root Pitch'); grid on;
subplot(1,2,2);
plot(theta_range, FM_th, 'k-s', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('\theta_0 [deg]'); ylabel('Figure of Merit');
title('FM vs Root Pitch'); grid on;
fprintf(' 4)Battery Cells\n');
Ncell_range = [3, 4, 5, 6];
GTOW_Nc = zeros(size(Ncell_range));
Mbat_Nc = zeros(size(Ncell_range));
for i = 1:length(Ncell_range)
    [d, conv] = design_quadrotor_winslow(R_base, Nb_base, c_base, ...
        theta0_base, Ncell_range(i), Kv_base, rho, g, Cla, Cd0, eta_m, ...
        r_cutout_frac, m_payload, m_electronics, t_endurance, N_rotors, ...
        xi_gl, w_gl, N_elem, V_cell, DOD, false);
    if conv, GTOW_Nc(i) = d.GTOW; Mbat_Nc(i) = d.m_battery;
    else, GTOW_Nc(i) = NaN; Mbat_Nc(i) = NaN; end
end

figure('Name','Study 4: Battery Cells','Position',[600 100 900 400]);
subplot(1,2,1);
bar(Ncell_range, GTOW_Nc*1000, 'FaceColor', [0.4 0.4 0.4]);
xlabel('Battery Cells (S)'); ylabel('GTOW [g]');
title('GTOW vs Battery Cells'); grid on;
subplot(1,2,2);
bar(Ncell_range, Mbat_Nc*1000, 'FaceColor', [0.6 0.6 0.6]);
xlabel('Battery Cells (S)'); ylabel('Battery Mass [g]');
title('Battery Mass vs Cells'); grid on;
fprintf('5) Blade Chord / AR\n');
c_range = linspace(0.006, 0.030, 30);
[GTOW_c, DL_c, FM_c, P_c] = sweep_param('c', c_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD);
AR_range = R_base ./ c_range;

figure('Name','Study 5: Chord/AR','Position',[50 300 1100 400]);
subplot(1,3,1);
plot(c_range*1000, GTOW_c*1000, 'k-o', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Chord [mm]'); ylabel('GTOW [g]');
title('GTOW vs Chord'); grid on;
```

```
subplot(1,3,2);
plot(AR_range, GTOW_c*1000, 'k-s', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Aspect Ratio (R/c)'); ylabel('GTOW [g]');
title('GTOW vs Blade AR'); grid on;
subplot(1,3,3);
sigma_range = Nb_base .* c_range ./ (pi * R_base);
plot(sigma_range, FM_c, 'k-^', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Solidity \sigma'); ylabel('Figure of Merit');
title('FM vs Solidity'); grid on;

fprintf(' 6) Endurance\n');
endur_range = linspace(5, 45, 30) * 60;
[GTOW_e, ~, ~, ~] = sweep_param('endurance', endur_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD);

figure('Name','Study 6: Endurance','Position',[600 300 450 400]);
plot(endur_range/60, GTOW_e*1000, 'k-o', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Endurance [min]'); ylabel('GTOW [g]');
title('GTOW vs Hover Endurance'); grid on;
fprintf('7) Payload\n');
payload_range = linspace(0.05, 1.0, 30);
[GTOW_p, ~, ~, ~] = sweep_param('payload', payload_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD);

figure('Name','Study 7: Payload','Position',[200 50 450 400]);
plot(payload_range*1000, GTOW_p*1000, 'k-o', 'LineWidth', 2, 'MarkerSize',
    3);
xlabel('Payload [g]'); ylabel('GTOW [g]');
title('GTOW vs Payload'); grid on;
Kv_range = linspace(700, 5000, 40);
[GTOW_p, ~, ~, ~] = sweep_param('Kv', Kv_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD);

figure('Name','Study 7: Kv','Position',[200 50 450 400]);
plot(Kv_range, GTOW_p*1000, 'k-o', 'LineWidth', 2, 'MarkerSize', 3);
xlabel('Kv'); ylabel('GTOW [g]');
title('GTOW vs Kv'); grid on;
R_sw     = linspace(0.08, 0.35, 6);
Nb_sw    = [2, 3, 4,5];
c_sw     = linspace(0.008, 0.021, 7);
th_sw    = linspace(14, 22, 10);
Nc_sw    = [3,4,5,6];
Kv_sw    = linspace(700,980,7);

best_GTOW = Inf;
best_design = [];
total = length(R_sw)*length(Nb_sw)*length(c_sw)*length(th_sw)*length(Nc_sw)
    *length(Kv_sw);
fprintf('Sweeping %d configurations...\n', total);

for i1 = 1:length(R_sw)
    for i2 = 1:length(Nb_sw)
```

```
        for i3 = 1:length(c_sw)
            for i4 = 1:length(th_sw)
                for i5 = 1:length(Nc_sw)
                    for i6 = 1:length(Kv_sw)
                        [d, conv] = design_quadrotor_winslow( ...
                            R_sw(i1), Nb_sw(i2), c_sw(i3), th_sw(i4), ...
                            Nc_sw(i5), Kv_sw(i6), rho, g, Cla, Cd0, ...
                            eta_m, r_cutout_frac, m_payload, m_electronics, ...
                            t_endurance, N_rotors, xi_gl, w_gl, N_elem, ...
                            V_cell, DOD, false);
                        if conv && d.GTOW < best_GTOW
                            if d.disc_loading >= 40 && d.disc_loading <= 90
                                best_GTOW = d.GTOW;
                                best_design = d;
                            end
                        end
                    end
                end
            end
        end
    end
end

fprintf('Optimization complete.\n\n');

if ~isempty(best_design)
    fprintf('OPTIMAL DESIGN \n');
    print_full_design(best_design);

    fprintf('\n\n BASELINE vs OPTIMAL \n');
    fprintf('%-35s %12s %12s\n', 'Parameter', 'Baseline', 'Optimal');
    fprintf('------------------------------------------------------------\n');
    if conv_base
        fprintf('%-35s %12.1f %12.1f\n', 'GTOW [g]', design_base.GTOW*1000, best_design.GTOW*1000);
        fprintf('%-35s %12.1f %12.1f\n', 'Rotor Radius [cm]', design_base.R*100, best_design.R*100);
        fprintf('%-35s %12d %12d\n', 'Blades/Rotor', design_base.Nb, best_design.Nb);
        fprintf('%-35s %12.1f %12.1f\n', 'Chord [mm]', design_base.c*1000, best_design.c*1000);
        fprintf('%-35s %12.1f %12.1f\n', 'Root Pitch [deg]', design_base.theta0_deg, best_design.theta0_deg);
        fprintf('%-35s %12d %12d\n', 'Battery Cells', design_base.N_cells, best_design.N_cells);
        fprintf('%-35s %12.0f %12.0f\n', 'Motor Kv', design_base.Kv, best_design.Kv);
        fprintf('%-35s %12.1f %12.1f\n', 'Disc Loading [N/m2]', design_base.disc_loading, best_design.disc_loading);
        fprintf('%-35s %12.0f %12.0f\n', 'Hover RPM', design_base.RPM_hover, best_design.RPM_hover);
        fprintf('%-35s %12.2f %12.2f\n', 'Hover Power/rotor [W]', design_base.P_shaft_hover, best_design.P_shaft_hover);
        fprintf('%-35s %12.4f %12.4f\n', 'Figure of Merit', design_base.FM, best_design.FM);
        fprintf('%-35s %12.1f %12.1f\n', 'Battery [g]', design_base.
```

```
    m_battery*1000, best_design.m_battery*1000);
        fprintf('%-35s %10.1f%%\n', 'GTOW Reduction', (1 - best_design.GTOW/design_base.GTOW)*100);
    end
    fprintf('--------------------------------------------------------------\n');
else
    fprintf('No valid design found within DL constraints (40-90 N/m2)!\n');
end


function [design, converged] = design_quadrotor_winslow( ...
    R, Nb, c, theta0_deg, N_cells, Kv, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD, verbose)

    design = struct();
    converged = false;

    sigma = Nb * c / (pi * R);
    A_disk = pi * R^2;
    AR_blade = R / c;
    r_cutout = r_cutout_frac;
    r_edges = linspace(r_cutout, 1, N_elem+1);

    % Linear twist
    theta_0_rad = deg2rad(theta0_deg);
    theta_tw_rad = deg2rad(-17);

    % BEMT
    [CT, CQ, CQi, CQp] = compute_CT_CQ(theta_0_rad, theta_tw_rad, ...
        sigma, Cla, Cd0, r_cutout, N_elem, r_edges, xi_gl, w_gl);

    if CT <= 0 || CQ <= 0
        if verbose, fprintf('Invalid CT or CQ. CT=%.6f, CQ=%.6f\n', CT, CQ); end
        return;
    end

    FM = CT^1.5 / (sqrt(2) * CQ);


    GTOW_old = 3 * m_payload * g;  % Initial guess
    tol_weight = 0.005 * g;         % 5 gram convergence
    max_iter = 300;

    for iter = 1:max_iter

        T_hover = GTOW_old / N_rotors;

        Omega_hover = sqrt(T_hover / (CT * rho * A_disk * R^2));
        RPM_hover   = Omega_hover * 60 / (2*pi);

        P_shaft_hover  = CQ * rho * A_disk * (Omega_hover * R)^3;
        P_elec_hover   = P_shaft_hover / eta_m;
        P_elec_tot_hov = N_rotors * P_elec_hover;

        V_batt = N_cells * V_cell;
```

```
        I_hover_tot = P_elec_tot_hov / V_batt;

        C_required_Ah = I_hover_tot * (t_endurance / 3600) / DOD;
        C_mAh = C_required_Ah * 1000;

        m_battery_g = 0.0418 * C_mAh^0.9327 * N_cells^1.0725;
        m_battery = m_battery_g / 1000;  % [kg]

        % 2. Design state (Twice Thrust -> Sizes Motors & ESC)

        T_design = 2 * T_hover;

        Omega_design   = sqrt(T_design / (CT * rho * A_disk * R^2));
        RPM_design     = Omega_design * 60 / (2*pi);

        P_shaft_design = CQ * rho * A_disk * (Omega_design * R)^3;
        P_elec_design  = P_shaft_design / eta_m;

        I_design_per_motor = P_elec_design / V_batt;


        R_cm = R * 100;
        m_rotor_g = 0.0195 * R_cm^2.0859 * sigma^(-0.2038) * Nb^0.5344;
        m_rotors_total = N_rotors * m_rotor_g / 1000;  % [kg]

        l_BL = 4.8910 * I_design_per_motor^0.1751 * P_shaft_design^0.2476;  % [mm]
        d_BL = 41.45 * Kv^(-0.1919) * P_shaft_design^0.1935;          % [mm]


        l_BL = max(l_BL, 1.1);
        d_BL = max(d_BL, 1.1);


        m_motor_g = 0.0109 * Kv^0.5122 * P_shaft_design^(-0.1902) * ...
                    (log10(l_BL))^2.5582 * (log10(d_BL))^12.8502;
        m_motor_g = max(m_motor_g, 2);  % minimum 2g motor
        m_motors_total = N_rotors * m_motor_g / 1000;  % [kg]

        m_esc_g = 0.8013 * I_design_per_motor^0.9727;
        m_esc_g = max(m_esc_g, 1);  % minimum 1g
        m_esc_total = N_rotors * m_esc_g / 1000;  % [kg]

        m_sub = m_payload + m_battery + m_rotors_total + m_motors_total + ...
                m_esc_total + m_electronics;

        m_airframe_g = 1.3119 * R_cm^1.2767 * m_battery_g^0.4587;
        m_airframe = m_airframe_g / 1000;  % [kg]

        m_total = m_sub + m_airframe;
        GTOW_new = m_total * g;

        if abs(GTOW_new - GTOW_old) < tol_weight
            converged = true;

            design.GTOW = m_total;
```

```
            design.GTOW_N = GTOW_new;
            design.m_payload = m_payload;
            design.m_battery = m_battery;
            design.m_battery_g = m_battery_g;
            design.m_rotors = m_rotors_total;
            design.m_rotor_each_g = m_rotor_g;
            design.m_motors = m_motors_total;
            design.m_motor_each_g = m_motor_g;
            design.m_esc = m_esc_total;
            design.m_esc_each_g = m_esc_g;
            design.m_electronics = m_electronics;
            design.m_airframe = m_airframe;
            design.m_airframe_g = m_airframe_g;

            design.R = R;
            design.Nb = Nb;
            design.c = c;
            design.sigma = sigma;
            design.AR_blade = AR_blade;
            design.theta0_deg = theta0_deg;
            design.N_cells = N_cells;
            design.Kv = Kv;

            design.CT = CT;
            design.CQ = CQ;
            design.CQi = CQi;
            design.CQp = CQp;
            design.FM = FM;

            % Hover specs
            design.RPM_hover = RPM_hover;
            design.vtip_hover = Omega_hover * R;
            design.T_hover_per_rotor = T_hover;
            design.P_shaft_hover = P_shaft_hover;
            design.I_hover_tot = I_hover_tot;

            % Design specs (Motor/ESC sizing metrics)
            design.T_design_per_rotor = T_design;
            design.RPM_design = RPM_design;
            design.P_shaft_design = P_shaft_design;
            design.I_design_per_motor = I_design_per_motor;

            design.V_batt = V_batt;
            design.C_mAh = C_mAh;
            design.l_BL = l_BL;
            design.d_BL = d_BL;
            design.disc_loading = GTOW_new / (N_rotors * A_disk);
            design.iterations = iter;

            if verbose
                fprintf('Converged in %d iterations. GTOW = %.1f g\n\n',
    iter, m_total*1000);
            end
            return;
        end

        GTOW_old = GTOW_new;
    end
```

```matlab

    if verbose
        fprintf('WARNING: Did not converge after %d iterations.\n', max_iter);
    end
end

function print_full_design(d)
    fprintf('\n==\n');
    fprintf('  OUTPUT 1: WEIGHT BREAKDOWN\n');
    fprintf('==\n');
    fprintf('%-35s %10.1f g  (%5.1f%%)\n', 'Payload', d.m_payload*1000, d.m_payload/d.GTOW*100);
    fprintf('%-35s %10.1f g  (%5.1f%%)\n', 'Battery', d.m_battery*1000, d.m_battery/d.GTOW*100);
    fprintf('%-35s %10.1f g  (%.1fg each x4, %4.1f%%)\n', 'Rotors (props)', ...
        d.m_rotors*1000, d.m_rotor_each_g, d.m_rotors/d.GTOW*100);
    fprintf('%-35s %10.1f g  (%.1fg each x4, %4.1f%%)\n', 'Motors (BLDC)', ...
        d.m_motors*1000, d.m_motor_each_g, d.m_motors/d.GTOW*100);
    fprintf('%-35s %10.1f g  (%.1fg each x4, %4.1f%%)\n', 'ESCs', ...
        d.m_esc*1000, d.m_esc_each_g, d.m_esc/d.GTOW*100);
    fprintf('%-35s %10.1f g  (%5.1f%%)\n', 'Electronics (FC+GPS+Rx)', ...
        d.m_electronics*1000, d.m_electronics/d.GTOW*100);
    fprintf('%-35s %10.1f g  (%5.1f%%)\n', 'Airframe', ...
        d.m_airframe*1000, d.m_airframe/d.GTOW*100);
    fprintf('---------------------------------------------------------\n');
    fprintf('%-35s %10.1f g\n', '*** TOTAL GTOW ***', d.GTOW*1000);
    fprintf('%-35s %10.2f N\n', '*** GTOW (force) ***', d.GTOW_N);

    fprintf('\n=========================================================\n');
    fprintf('  OUTPUT 2: GEOMETRIC & PHYSICAL SPECIFICATIONS\n');
    fprintf('=========================================================\n');
    fprintf('%-35s %10.1f cm  (%.1f inch)\n', 'Rotor Radius', d.R*100, d.R*100/2.54);
    fprintf('%-35s %10.1f cm  (%.1f inch)\n', 'Prop Diameter', d.R*200, d.R*200/2.54);
    fprintf('%-35s %10.1f mm\n', 'Blade Chord', d.c*1000);
    fprintf('%-35s %10d\n', 'Blades per Rotor', d.Nb);
    fprintf('%-35s %10.4f\n', 'Solidity (sigma)', d.sigma);
    fprintf('%-35s %10.1f\n', 'Blade Aspect Ratio (R/c)', d.AR_blade);
    fprintf('%-35s %10.1f deg\n', 'Root Pitch', d.theta0_deg);
    fprintf('%-35s %10s\n', 'Twist', '-17 deg/R (linear)');

    fprintf('\n--- Motor & Electrical (Sized for 2x Thrust) ---\n');
    fprintf('%-35s %10.0f kV\n', 'Motor Kv', d.Kv);
    fprintf('%-35s %10.1f mm\n', 'Motor Length (est.)', d.l_BL);
    fprintf('%-35s %10.1f mm\n', 'Motor Diameter (est.)', d.d_BL);
    fprintf('%-35s %10.1f A\n', 'Design Current/Motor (2x Thrust)', d.I_design_per_motor);
    fprintf('%-35s %10.1f W\n', 'Design Power/Motor (2x Thrust)', d.P_shaft_design);

    fprintf('\n--- Battery ---\n');
    fprintf('%-35s %10d S\n', 'Cells', d.N_cells);
    fprintf('%-35s %10.1f V\n', 'Voltage', d.V_batt);
```

```
    fprintf('%-35s %10.0f mAh\n', 'Capacity', d.C_mAh);
    fprintf('%-35s %10.1f A\n', 'Hover Current Draw (Total)', d.I_hover_tot);

    fprintf('\n============================================================\n');
    fprintf('  ROTOR PERFORMANCE (BEMT)\n');
    fprintf('============================================================\n');
    fprintf('%-35s %10.6f\n', 'CT', d.CT);
    fprintf('%-35s %10.6f\n', 'CQ', d.CQ);
    fprintf('%-35s %10.4f\n', 'Figure of Merit', d.FM);
    fprintf('%-35s %10.1f N/m^2\n', 'Disc Loading (Hover)', d.disc_loading);

    fprintf('\n--- HOVER STATE ---\n');
    fprintf('%-35s %10.0f RPM\n', 'Hover RPM', d.RPM_hover);
    fprintf('%-35s %10.2f N\n', 'Hover Thrust per Rotor', d.T_hover_per_rotor);
    fprintf('%-35s %10.2f W\n', 'Hover Shaft Power/Rotor', d.P_shaft_hover);

    fprintf('\n--- DESIGN STATE (2x Hover Thrust) ---\n');
    fprintf('%-35s %10.0f RPM\n', 'Design RPM', d.RPM_design);
    fprintf('%-35s %10.2f N\n', 'Design Thrust per Rotor', d.T_design_per_rotor);
    fprintf('%-35s %10.2f W\n', 'Design Shaft Power/Rotor', d.P_shaft_design);

    fprintf('\n%-35s %10d\n', 'Convergence Iterations', d.iterations);
    fprintf('============================================================\n');
end

function [GTOW_arr, DL_arr, FM_arr, P_arr] = sweep_param(param_name, param_range, ...
    R_base, Nb_base, c_base, theta0_base, Ncell_base, Kv_base, ...
    rho, g, Cla, Cd0, eta_m, r_cutout_frac, m_payload, m_electronics, ...
    t_endurance, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD)

    n = length(param_range);
    GTOW_arr = NaN(1, n);
    DL_arr   = NaN(1, n);
    FM_arr   = NaN(1, n);
    P_arr    = NaN(1, n);

    for i = 1:n
        R_i = R_base; Nb_i = Nb_base; c_i = c_base;
        th_i = theta0_base; Nc_i = Ncell_base; Kv_i = Kv_base;
        mp_i = m_payload; te_i = t_endurance;

        switch param_name
            case 'R',         R_i  = param_range(i);
            case 'Nb',        Nb_i = param_range(i);
            case 'c',         c_i  = param_range(i);
            case 'theta',     th_i = param_range(i);
            case 'Ncell',     Nc_i = param_range(i);
            case 'Kv',        Kv_i = param_range(i);
            case 'endurance', te_i = param_range(i);
            case 'payload',   mp_i = param_range(i);
```

```
        end

        [d, conv] = design_quadrotor_winslow(R_i, Nb_i, c_i, th_i, Nc_i, Kv_i, ...
            rho, g, Cla, Cd0, eta_m, r_cutout_frac, mp_i, m_electronics, ...
            te_i, N_rotors, xi_gl, w_gl, N_elem, V_cell, DOD, false);

        if conv
            GTOW_arr(i) = d.GTOW;
            DL_arr(i)   = d.disc_loading;
            FM_arr(i)   = d.FM;
            P_arr(i)    = d.P_shaft_hover * N_rotors / eta_m;
        end
    end
end

function [CT, CQ, CQi, CQp] = compute_CT_CQ(theta0, theta_tw, sigma, ...
    Cla, Cd0, r_cutout, N_elem, r_edges, xi_gl, w_gl)

    CT = 0; CQ = 0; CQi = 0; CQp = 0;

    for e = 1:N_elem
        r_a = r_edges(e);
        r_b = r_edges(e+1);
        dr = r_b - r_a;

        for g_idx = 1:6
            r_bar = 0.5*(r_a + r_b) + 0.5*dr*xi_gl(g_idx);

            if r_bar < r_cutout || r_bar < 1e-6
                continue;
            end

            theta_local = theta0 + theta_tw * r_bar;

            a_q = 4;
            b_q = sigma * Cla / 2;
            c_q = -sigma * Cla * theta_local * r_bar / 2;

            disc = b_q^2 - 4*a_q*c_q;
            if disc < 0
                continue;
            end

            lam = (-b_q + sqrt(disc)) / (2*a_q);
            lam = max(lam, 0);

            dCT_val  = 0.5 * sigma * Cla * (theta_local * r_bar^2 - lam * r_bar);
            dCQi_val = lam * dCT_val;
            dCQp_val = 0.5 * sigma * Cd0 * r_bar^3;
            dCQ_val  = dCQi_val + dCQp_val;

            wt = w_gl(g_idx) * 0.5 * dr;
            CT  = CT  + wt * dCT_val;
            CQ  = CQ  + wt * dCQ_val;
            CQi = CQi + wt * dCQi_val;
```

```
                CQp = CQp + wt * dCQp_val;
            end
        end
    end
```

Listing 1: Main script: parametric sweeps and optimisation.